Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (B.Sc.I.T Semester V)
Full Stack development MERN Practical

Roll No.: T025	Name: Afsha Amjad Mujawar
Class: TYIT	Batch:1
Date of Assignment: 12-10-2026	Date/Time of Submission: 12-10-2026

Practical- 10

Aim: Full Stack Execution and Validation:

a. Integrate React frontend with Node and Express backend.
b. Perform insert, update, and delete operations through UI.
c. Display database records dynamically in React.
d. Implement frontend form validation.
e. Test all APIs using Postman.
f. Execute and verify the complete working of the frontend, backend, and database together.

Code:

- **server.js :**

```
const express = require("express");
const mongoose = require("mongoose");
const cors = require("cors");
const app = express();
app.use(cors());
app.use(express.json());
// Connect to MongoDB
mongoose.connect("mongodb://127.0.0.1:27017/CollegeDB")
.then(() => {
console.log("MongoDB Connected Successfully");
app.listen(3000, () => {
console.log("Server running on port 3000");
});
})
.catch((error) => {
console.log("MongoDB Connection Failed:", error.message);
});

// Student model with validation
const Student = mongoose.model("Student", {
name: {
type: String,
required: [true, "Student name is required"],
trim: true
},
course: {
type: String,
required: [true, "Course is required"],
trim: true
},
marks: {
type: Number,
required: [true, "Marks are required"],
min: [0, "Marks cannot be less than 0"],
max: [100, "Marks cannot exceed 100"]
}
});
// Validate student data
function validateStudent(data) {
const { name, course, marks } = data;
if (
typeof name !== "string" ||
!name.trim() ||
typeof course !== "string" ||
!course.trim() ||
marks === "" ||
marks === undefined ||
!Number.isFinite(Number(marks)) ||
Number(marks) < 0 ||
Number(marks) > 100
) {
return false;
}
return true;

}
// GET: Display all students
app.get("/students", async (req, res) => {
try {
const students = await Student.find();
res.json(students);
} catch (error) {
res.status(500).json({ message: "Error fetching students" });
```

```
}
});
// POST: Add a student
app.post("/students", async (req, res) => {
if (!validateStudent(req.body)) {
return res.status(400).json({
message: "Enter a valid name, course and marks
between 0 and 100"
});
}
try {
const student = await Student.create({
name: req.body.name.trim(),
course: req.body.course.trim(),
marks: Number(req.body.marks)
});
res.status(201).json({
message: "Student added successfully",
student
});
} catch (error) {
res.status(400).json({ message: "Unable to add
student" });
}
});
// PUT: Update a student
app.put("/students/:id", async (req, res) => {
if (!mongoose.isValidObjectId(req.params.id)) {
return res.status(400).json({ message: "Invalid
student ID" });
}
if (!validateStudent(req.body)) {
return res.status(400).json({
message: "Enter a valid name, course and marks
between 0 and 100"
});
}
try {
const student = await Student.findByIdAndUpdate(
req.params.id,
{
name: req.body.name.trim(),
course: req.body.course.trim(),
marks: Number(req.body.marks)
},
{ new: true, runValidators: true }
);
if (!student) {
return res.status(404).json({ message: "Student not
found" });
}
res.json({
message: "Student updated successfully",
student
});
} catch (error) {
res.status(400).json({ message: "Unable to update
student" });
}
});
// DELETE: Delete a student
app.delete("/students/:id", async (req, res) => {
if (!mongoose.isValidObjectId(req.params.id)) {
return res.status(400).json({ message: "Invalid
student ID" });
}
try {
const student = await
Student.findByIdAndDelete(req.params.id);

if (!student) {
return res.status(404).json({ message: "Student not
found" });
}
res.json({ message: "Student deleted successfully"
});
} catch (error) {
res.status(500).json({ message: "Error deleting
student" });
}
});
```

- **App.jsx**

```
import { useEffect, useState } from "react";
import axios from "axios";
const API = "http://localhost:3000/students";
function App() {
const [students, setStudents] = useState([]);
const [name, setName] = useState("");
const [course, setCourse] = useState("");
const [marks, setMarks] = useState("");
const [editId, setEditId] = useState(null);
const [error, setError] = useState("");
// Get student records
const getStudents = async () => {
try {
const response = await axios.get(API);
setStudents(response.data);

setError("");
} catch (err) {
setError("Unable to load students. Check the
backend.");
}
};
useEffect(() => {
getStudents();
}, []);
// Clear form
const clearForm = () => {
setName("");
```

```
setCourse("");
setMarks("");
setEditId(null);
setError("");
};
// Add or update student
const saveStudent = async (e) => {
e.preventDefault();
setError("");
// Validation 1: Empty fields
if (!name.trim() || !course.trim() || marks ===
"") {
setError("Please fill in all fields.");
return;
}
// Validation 2: Marks must be between 0 and
100
if (
!Number.isFinite(Number(marks)) ||
Number(marks) < 0 ||
Number(marks) > 100
) {
setError("Marks must be between 0 and
100.");
return;
}
const student = {
name: name.trim(),
course: course.trim(),
marks: Number(marks)
};
try {
let response;
if (editId) {
response = await
axios.put(`${API}/${editId}`, student);
} else {
response = await axios.post(API, student);
}
alert(response.data.message);
clearForm();
await getStudents();
} catch (err) {
setError(
err.response?.data?.message || "Unable to save
student."
);
}
};
// Load details for editing
const editStudent = (student) => {
setEditId(student._id);
setName(student.name);
setCourse(student.course);
setMarks(String(student.marks));
setError("");
};
// Delete student
const deleteStudent = async (id) => {
if (!window.confirm("Do you want to delete
this student?")) {
return;
}

try {
const response = await
axios.delete(`${API}/${id}`);
alert(response.data.message);
if (editId === id) {
clearForm();
}
await getStudents();
} catch (err) {
setError(
err.response?.data?.message || "Unable to
delete student."
);
}
};
return (
<div>
<h1>Student Management System</h1>
<h2>{editId ? "Update Student" : "Add
Student"}</h2>
<form onSubmit={saveStudent}>
<input
type="text"
placeholder="Student Name"
value={name}
onChange={(e) => setName(e.target.value)}
/>
<input
type="text"
placeholder="Course"
value={course}
onChange={(e) => setCourse(e.target.value)}
/>
<input
type="number"
placeholder="Marks (0-100)"

min="0"
max="100"
step="any"
value={marks}
onChange={(e) => setMarks(e.target.value)}
/>
<button type="submit">
{editId ? "Update Student" : "Add Student"}
</button>
```

```
{editId && (
<button type="button" onClick={clearForm}>
Cancel

</button>
)}
</form>
{error && <p style={{ color: "red"
}}>{error}</p>}
<h2>Student List</h2>
<table border="1" cellPadding="8">
<thead>
<tr>
<th>Name</th>
<th>Course</th>
<th>Marks</th>
<th>Actions</th>
</tr>
</thead>
<tbody>
{students.length === 0 ? (
<tr>
<td colSpan="4">No students available.</td>
</tr>
) : (
students.map((student) => (
<tr key={student._id}>

<td>{student.name}</td>
<td>{student.course}</td>
<td>{student.marks}</td>
<td>
<button
type="button"
onClick={() => editStudent(student)}
>
Edit
</button>
<button
type="button"
onClick={() => deleteStudent(student._id)}
>
Delete
</button>
</td>
</tr>
))
)}
</tbody>
</table>
</div>
);
}
export default App;
```

Output:

